

72. Une fonction qui prend des arguments

Écrire une fois ce qu'on fera dix fois.

1. Ce qu'il faut comprendre

Voici ce que le C++ apporte de plus visible. Une fonction prend des **arguments**, et rend une **valeur** : `uint8_t distance(uint8_t a, uint8_t b)`. On l'écrit une fois, on l'appelle partout, et l'endroit où l'on écrit `barre(2, 6, 5)` dit exactement ce qui va se passer.

Une fonction qui ne rend rien s'écrit `void`. Une fonction qui annonce rendre un `uint8_t` **doit** le faire : si aucun `return` ne rend de valeur, le compilateur refuse le fichier, plutôt que de laisser la fonction rendre ce qui traînait dans le processeur.

Un prix à payer, et il est dit franchement : **une fonction ne peut pas s'appeler elle-même**. Les arguments vivent à une place fixe, réservée une fois pour toutes — faute d'une pile praticable sur cette machine. Un appel imbriqué écraserait les arguments de l'appel en cours ; le compilateur nomme le cycle et s'arrête là.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **546 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
distance()	23 o	← main() — ta fonction
barre()	38 o	← main() — ta fonction
main()	73 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	76 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	176 o	← CopierTuiles — 10 lettres sur 43, et 1 tuile dessinée
"DES ARGUMENTS"	13 o	← texte() — un texte gravé

Lettres gravées : ADEGMNRSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	x 1	17 o sur place, dans barre() + AdresseCase (10 o)
texte()	x 1	11 o sur place, dans main() + EcrireTexte (10 o)

```
image()    x 1    3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
barre()    x 3    43 o sur place, dans main()
distance() x 1    11 o sur place, dans main()
```

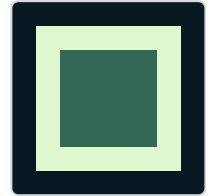
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <poser>    // pose une tuile sur une case du fond
#include <texte>    // écrit un texte à l'écran
```

2. UN DESSIN — BLOC

```
Tuile BLOC = {
    "33333333",
    "30000003",
    "30222203",
    "30222203",
    "30222203",
    "30222203",
    "30000003",
    "33333333",
};

// Une fonction qui rend une valeur.
```



à l'écran

3. UNE FONCTION — DISTANCE()

```
uint8_t distance(uint8_t a, uint8_t b) {
    if (a > b) return a - b;
    return b - a;
}

// Une fonction qui n'en rend aucune : « void ».
```

4. UNE FONCTION — BARRE()

```
void barre(uint8_t colonne, uint8_t ligne, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, BLOC);
    }
}
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(2, 2, "DES ARGUMENTS");

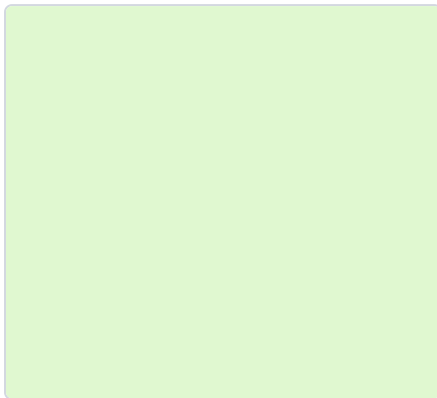
    barre(2, 6, 5);
    barre(2, 8, 12);
    barre(2, 10, distance(3, 20));

    while (true) {
        image();
    }

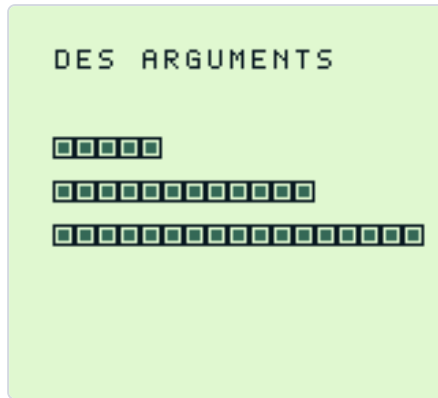
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois barres de longueurs différentes : 5, 12, puis 17 cases.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première barre fait 5 cases (5)
- ✓ la deuxième en fait 12 (12)
- ✓ la troisième vaut `distance(3, 20)`, soit 17 (17)
- ✓ le titre est écrit

Ensuite : **73. Découper son programme** — chapitre 11 : Découper son programme. · gameboy3 — écrire une cartouche Game Boy en C++